

第 11 讲：Forward Propagation 与 Computational Graph

一次预测到底是怎样一步步算出来的

真正理解 Forward

Propagation（前向传播）是什么：从输入开始，按依赖顺序计算所有中间量，直到得到输出。

把上一讲的 MLP 展开成 Computational Graph（计算图），看清“谁依赖谁”。

学会区分网络结构图、计算图、Forward Pass 三个概念。

理解中间值为什么必须被记录：后续 Loss、Gradient、Chain Rule、Backpropagation 都依赖这些关系。

实验：标量链、两层 MLP、输入扰动、ReLU 截断路径、NumPy / PyTorch 对照。

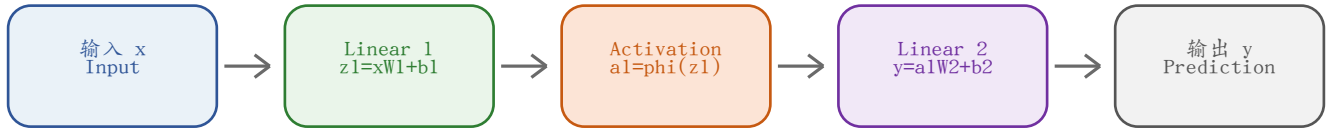
本讲定位

这是后面反向传播之前最重要的“地图课”。Backpropagation 不是凭空出现的算法；它是在 Forward 已经建立好的依赖图上，反方向计算梯度。

1. 【原理】Forward Propagation 到底是什么？

Forward Propagation（前向传播，简称 Forward Pass）就是：给定输入和当前参数以后，从网络的第一步开始，按计算依赖顺序一直算到最终输出。

Forward：数字从左向右流动



“传播”这个词容易让初学者产生神秘感。实际上它并不是某种额外算法，而是把已经定义好的数学函数依次执行。

一句话

Forward = 用当前 Weight / Bias，把输入一步一步算成输出。

训练阶段和推理阶段都会做 Forward。训练时 Forward 后面还会继续计算 Loss 和 Backward；推理时通常得到输出后就结束。

2. 【通俗解释】把神经网络想成一条“计算流水线”

假设一个工厂流水线有 4 个工位：第一个工位把原料混合，第二个工位筛选，第三个工位再次组合，最后得到产品。神经网络的 Forward 也类似：每个节点只负责执行自己的计算，然后把结果交给下一个节点。

计算阶段	输入	内部发生什么	输出
Linear 1	x	矩阵乘法 + Bias	z1
Activation	z1	逐元素非线性变换	a1
Linear 2	a1	矩阵乘法 + Bias	y

关键区别
网络“结构”说明有哪些层；Forward 说明这些层在一次输入上实际算出了哪些数字。

所以看 PyTorch 代码时，`model(x)` 并不是一句“让模型思考”的魔法命令。它触发的是一连串具体 Tensor 运算。

3. 【具体数字例子】一个最小标量 Forward

先不用矩阵，使用一个只有标量的计算链：

$$z = 2x - 1$$

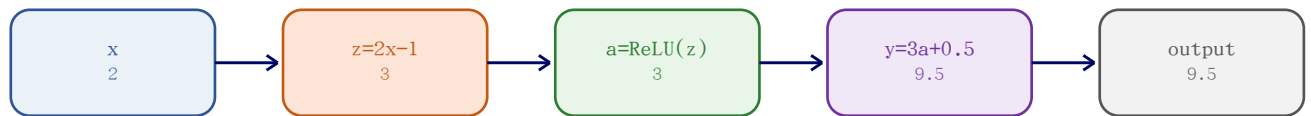
$$a = \text{ReLU}(z)$$

$$y = 3a + 0.5$$

令 $x = 2$ ，则：

步骤	公式	代入数字	结果
1	$z = 2x - 1$	$2*2 - 1$	$z = 3$
2	$a = \text{ReLU}(z)$	$\text{ReLU}(3)$	$a = 3$
3	$y = 3a + 0.5$	$3*3 + 0.5$	$y = 9.5$

一个最小 Forward Computational Graph



注意：每个中间量不仅有一个数值，还带着“我是由谁算出来的”这层依赖关系。计算图就是把这种依赖明确记录下来。

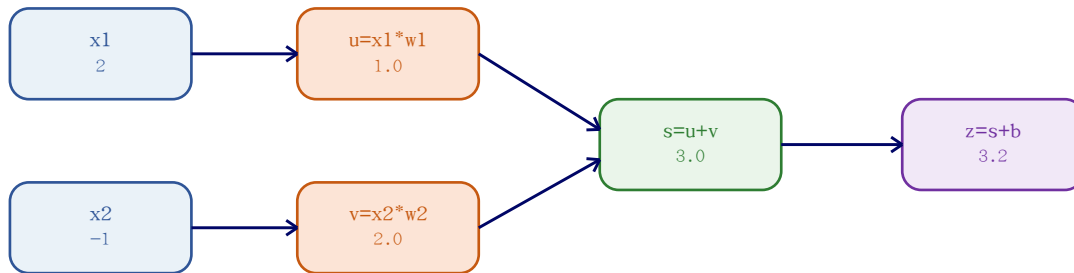
4. 【原理】Computational Graph 是什么？

Computational Graph（计算图）把一个复杂表达式拆成很多简单操作。图中的节点可以表示 Tensor / 中间变量，也可以表示操作；边表示依赖关系。

例如一个神经元：

$$z = x1 * w1 + x2 * w2 + b$$

把 Dot Product 拆成依赖关系图



这里的 z 依赖 s , s 又依赖 u 和 v , u 依赖 $x1$ 与 $w1$, v 依赖 $x2$ 与 $w2$ 。Forward 是沿这些依赖从“已知输入”向“最终输出”求值。

为 Backprop 做准备

后面求 $dz/dx1$ 时，不需要重新发明规则，只要沿着 $x1 \rightarrow u \rightarrow s \rightarrow z$ 这条依赖路径反向使用 Chain Rule。

5. 【不要混淆】结构图、计算图、Forward Pass

概念	它回答的问题	例子
Network Architecture / 结构图	模型由哪些模块组成?	Linear -> ReLU -> Linear
Computational Graph / 计算图	具体变量和操作之间谁依赖谁?	$x \rightarrow z1 \rightarrow a1 \rightarrow y$
Forward Pass / 前向传播	给定这次输入, 实际算出的值是多少?	$x=[1,2,-1] \rightarrow y=[2.7,0.2]$

同一个模型结构, 在不同输入上会得到不同的 Forward 中间值。结构通常不变, 但一次 Forward 中的具体 Tensor 数值会变化。

初学者常见误区

“计算图”不是模型结构图的漂亮版本。它的核心价值是表示数学依赖, 因此后面自动求导才能知道梯度应该沿哪些路径传播。

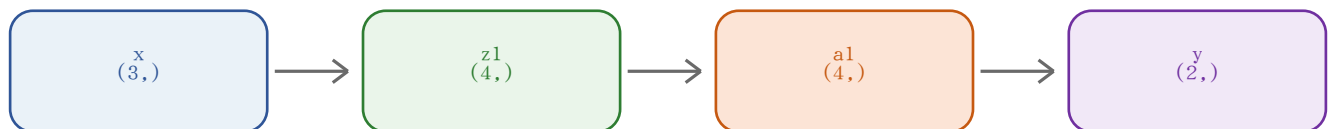
6. 【回到上一讲】3 → 4 → 2 MLP 的完整 Forward

继续使用第 10 讲已经手算过的小网络：

$$\mathbf{x} = [1, 2, -1]$$

阶段	公式 / 操作	Shape	本例数值
Input	$\mathbf{x}$	(3,)	[1, 2, -1]
Linear 1	$\mathbf{z1} = \mathbf{xW1} + \mathbf{b1}$	(4,)	[1, 3, -1, -3]
ReLU	$\mathbf{a1} = \text{ReLU}(\mathbf{z1})$	(4,)	[1, 3, 0, 0]
Linear 2	$\mathbf{y} = \mathbf{a1W2} + \mathbf{b2}$	(2,)	[2.7, 0.2]

这一整条链就是一次 Forward Pass



这一讲的新重点不是重新算 MLP，而是看清： $\mathbf{z1}$ 、 $\mathbf{a1}$ 都是计算图中的中间节点，后续 Loss 和 Backward 会使用这些依赖。

7. 【Shape 视角】Forward 不只是算数值，还要追踪 Tensor 结构

对深度学习科研而言，Forward 时必须同时追踪两条信息：

数值发生了什么变化？

Shape 发生了什么变化？

例如 LLM 风格的 Tensor：

$$X : (B, T, D)$$

如果一个 FFN 的第一层把隐藏维 D 扩张到 H：

$$(B, T, D) @ (D, H) \rightarrow (B, T, H)$$

Activation 逐元素处理：

$$(B, T, H) \rightarrow (B, T, H)$$

第二层再投影回 D：

$$(B, T, H) @ (H, D) \rightarrow (B, T, D)$$

与 Transformer 的连接

Transformer Block 中的 FFN / MLP Forward 本质上就是这样的 Tensor 流。看到结构图时，应主动在每条边上补上 Shape。

8. 【代码实验】NumPy：把每一个中间量显式写出来

```
import numpy as np

x = np.array([1.0, 2.0, -1.0])

# ■■■■Linear
z1 = x @ W1 + b1

# ■■■■ReLU
a1 = np.maximum(0.0, z1)

# ■■■■Linear
y = a1 @ W2 + b2

print('x :', x, x.shape)
print('z1:', z1, z1.shape)
print('a1:', a1, a1.shape)
print('y :', y, y.shape)
```

这种写法虽然比 `model(x)` 嗦，但非常适合初学阶段：每一个中间 Tensor 都可见。后面进入 PyTorch 后，你仍然应该保留“我知道里面有哪些中间节点”的心智模型。

学习原则

先学会把黑盒拆开，再学习如何让框架把这些步骤封装起来。

9. 【实验直觉】改变输入，Forward 的影响沿依赖关系向后传播

Forward Graph

同时告诉我们一个非常朴素的事实：如果某个输入改变，只有依赖它的后续节点才可能改变。

例如：

$$x \rightarrow z1 \rightarrow a1 \rightarrow y$$

把 x 的一个分量从 1.0 改成 1.1，首先会改变第一层 $z1$ ；这些变化再经过 ReLU，得到新的 $a1$ ；最后才可能改变 y 。

注意

这里还没有计算 Gradient。我们只是做两次

Forward，然后比较两个输出。这属于“有限扰动观察”，不是导数。后面学习 Gradient

时，我们会研究“无限小变化下的变化率”。

这个思想以后做科研也很重要：Ablation、输入扰动、噪声实验，本质上都在观察一个系统的依赖路径如何把变化传到输出。

10. 【一个关键现象】ReLU 可以让某条 Forward 路径暂时“关掉”

考虑非常简单的一条路径：

$$z \rightarrow \text{ReLU}(z) \rightarrow y$$

z	ReLU(z)	如果 $y = 2 * \text{ReLU}(z)$	结果
-1.0	0	0	路径被截断
-0.5	0	0	z 变了，但下游不变
0.5	0.5	1.0	路径开始传递数值
2.0	2.0	4.0	继续向下游传播

这正是非线性函数改变信息流的一个直观例子。后面讲 Gradient 时，你还会看到 ReLU 在负区间的局部梯度为 0，于是 Backward 也会受到影响。

先记住现象，不抢跑公式
现在只观察 Forward：负值经过 ReLU 变成 0。第 13 讲以后再学习导数，第 22 讲 Chain Rule 后再把这件事放进 Backpropagation。

11. 【PyTorch 连接】model(x) 做了什么？

```
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(3, 4),
    nn.ReLU(),
    nn.Linear(4, 2)
)

x = torch.tensor([1.0, 2.0, -1.0])
y = model(x)
```

`model(x)` 会按照 `nn.Sequential` 中模块的顺序依次执行。概念上就是：

PyTorch 封装后的 Forward



PyTorch 还会在需要梯度时记录运算关系，为 Autograd 建立动态计算图。这个机制我们会在第 29 讲专门学习；当前只需要知道：框架并没有改变数学，只是帮我们执行并记录。

不要过早依赖黑盒

现在如果你只会 `y=model(x)`，但说不出内部经过哪些 Shape、哪些运算，那么基础还没有闭环。

12. 【与 LLM / Transformer 的关系】一次 Token Forward 是怎样向前走的？

你以前在 LLM 课程见过下面的整体链条：

LLM Forward (极度简化)



现在应该换一种理解方式：这不是 5 个“神秘模块”，而是一张巨大的 Computational Graph。里面包含矩阵乘法、归一化、Attention、MLP、Residual 等大量 Tensor 运算。

训练时还会继续：

Logits -> Cross Entropy Loss -> Backpropagation

本讲与以后课程的桥梁

第 18 讲学 Cross Entropy 后，我们会把 Forward 延伸到 Loss；第 22-26 讲再沿整张图反向传播 Gradient。

13. 【常见误区】本讲最容易理解错的地方

误区 1: Forward 就是“预测”

不完全。Forward

是从输入到输出的计算过程；输出可以是预测、隐藏表示、Logits，甚至某个中间模块的结果。

误区 2: 计算图就是网络结构图

结构图偏模块级；计算图强调数学依赖，通常更细。Autograd 真正关心的是“这个 Tensor 是由哪些运算得到的”。

误区 3: Forward 只在推理时发生

错误。训练每个 Step 都先 Forward 得到预测与 Loss，然后才 Backward。

误区 4: 中间 Tensor 没什么用

错误。中间值决定后续输出；训练时很多中间信息还与梯度计算有关。

误区 5: Forward Propagation 和 Backpropagation 是两套互不相关的算法

错误。Backward 正是沿 Forward 建立的依赖关系反方向计算敏感度 / 梯度。

14. 【科研视角】为什么做实验时要学会画自己的 Forward 数据流？

以后阅读或设计 Agent Systems / LLM 实验时，即使你的研究重点不在训练基础模型，也经常要理解系统内部的模型调用、Embedding、Projection、Loss 或评分模块。一个可靠方法是把复杂流程拆成“输入 - 中间量 - 输出”。

科研问题	Forward / 计算图能帮你看什么？
某个组件为什么影响最终指标？	先找它到输出的依赖路径
Ablation 删除了什么？	看被删除节点及其下游路径
Shape 报错从哪里来？	沿 Forward 一步一步记录每个 Tensor Shape
结果异常但程序不报错？	打印关键中间量，判断异常从哪一步开始
复现实验与论文结构不一致？	把论文模块图映射到实际代码的 Forward 路径
科研习惯 遇到复杂模型，不要第一反应就读完整源码。先画最小 Forward 数据流，再逐层进入实现细节。	

15. 【本讲总结】你应该真正留下什么？

Forward Propagation: 从输入开始，按照定义好的运算顺序计算中间量，最终得到输出。

Computational Graph: 记录变量、操作及其依赖关系。

一次 Forward 要同时关注：数值、Shape、依赖关系。

训练和推理都会 Forward；训练时 Forward 后继续计算 Loss 和 Backward。

PyTorch 的 `model(x)` 只是把多个 Forward 操作封装起来。

未来 Backpropagation 就是在这张依赖图上反方向使用 Chain Rule。

最低验收标准

看到 `x -> Linear -> ReLU -> Linear -> y` 时，你能够把每一步写成公式、说出输入输出 Shape，并解释每个中间量依赖谁。

下一讲：第 12 讲《第一次完整 NumPy 神经网络 - 不用 PyTorch，把 Forward 从头写出来》

下一讲会把第 7-11 讲真正收束成一个可运行的小网络：参数由我们显式保存，Forward 函数由我们自己写，每一步打印 Shape，并与手算结果逐项核对。